

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

Introdução ao controle de versão

Aula 2

Código da aula: [DADOS]ANO1C1B1S4A2

Exposição



Objetivos da aula

- Relembrar o que é controle de versão e demonstrar as vantagens e as desvantagens das ferramentas disponíveis no mercado.



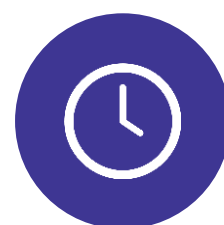
Competências da unidade (técnicas e socioemocionais)

- Usar ferramentas de desenvolvimento de software, como Git e GitHub;
- Trabalhar em equipe, compartilhando conhecimentos, contribuindo com ideias e colaborando para alcançar objetivos comuns.



Recurso didático

- Recurso audiovisual para a exibição de vídeos e imagens.



Duração da aula

50 minutos.

Relembrando: o que é controle de versão?

Controle de versão é um sistema que gerencia alterações no código-fonte e outros arquivos ao longo do tempo, permitindo o acompanhamento das modificações, a reversão a versões anteriores e a colaboração **eficiente** entre desenvolvedores.

Ele registra as alterações feitas em um projeto, facilitando o trabalho em equipe e a manutenção do histórico de desenvolvimento. O controle de versão é essencial para evitar conflitos, rastrear bugs e garantir a estabilidade e a integridade do software em evolução.

Métodos tradicionais de controle de versão

Métodos tradicionais de controle de versão, antes da era digital e do uso de sistemas automatizados, incluíam práticas como:

- cópias físicas de documentos ou manuscritos;
- cópias com datas e até número da versão;
- comentários no código com data;
- etiquetas.

Métodos tradicionais de controle de versão

Esses métodos tinham limitações significativas em termos de eficiência e rastreabilidade.

Encontrar uma versão específica ou rastrear alterações ao longo do tempo era desafiador, e a gestão de várias cópias físicas podia ser propensa a erros.

Problemas dos métodos tradicionais

O controle manual de versões, como cópias físicas de documentos, apresenta vários problemas, incluindo:

- dificuldade de rastreamento;
- propensão a erros;
- ineficiência na colaboração;
- consumo de espaço físico;
- ausência de histórico detalhado;
- dificuldade na reversão de alterações.

Controle de versão digital

O controle de versão digital é uma prática fundamental no desenvolvimento de software, permitindo o rastreamento eficiente de alterações no código-fonte e na documentação ao longo do tempo.

Ao contrário dos métodos manuais, o controle de versão digital oferece **precisão, facilidade de colaboração e histórico detalhado**, contribuindo para um desenvolvimento mais organizado e controlado.

Exposição

Ferramenta de controle de versão: Git



Reprodução – GIT. Disponível em:
<https://git-scm.com/downloads/logos>.
Acesso em: 16 jan. 2024.

O Git é um sistema de controle de versão distribuído desenvolvido em 2005 e amplamente utilizado na indústria de desenvolvimento de software.

Desenvolvido por Linus Torvalds, o criador do Linux, o Git oferece uma abordagem eficaz para o controle de versão, permitindo o acompanhamento de alterações em projetos de qualquer tamanho.

Sua natureza distribuída permite que cada desenvolvedor mantenha uma cópia local completa, facilitando a colaboração e reduzindo a dependência de uma conexão contínua à internet.

Surgem ferramentas inovadoras

O Git foi implementado por softwares como **GitLab** e **GitHub**, que adicionam mais possibilidades ao controle de versão.



Reprodução - GitLab. Disponível em: <https://about.gitlab.com/press/press-kit/>. Acesso em: 16 jan. 2024.



Reprodução - GitHub. Disponível em: <https://github.com/logos>. Acesso em: 16 jan. 2024.

GitHub e GitLab

O Git, por si só, é um sistema de controle de versão distribuído poderoso e eficiente. No entanto, ferramentas de hospedagem de repositórios como o GitHub e o GitLab complementaram o Git, proporcionando uma camada adicional de recursos e funcionalidades.

Nos slides a seguir, exploraremos como essas ferramentas aprimoram o uso do Git.

GitHub e GitLab

Colaboração remota: o Git, por natureza, permite que os desenvolvedores trabalhem localmente em seus próprios ramos (*branches*). O GitHub e o GitLab expandem essa capacidade ao permitir que equipes distribuídas colaborem de maneira eficiente. Desenvolvedores podem sincronizar seus repositórios locais com repositórios remotos, facilitando a colaboração em projetos de qualquer lugar do mundo.

Gerenciamento de problemas e tarefas: ambas as plataformas oferecem recursos de rastreamento de problemas (*issues*) e gerenciamento de tarefas integrados. Isso permite que equipes associem alterações de código a problemas específicos, acompanhem o progresso e facilitem a comunicação em torno de questões relacionadas ao desenvolvimento.

GitHub e GitLab

Revisão de código (*code review*): a revisão de código é uma prática crucial no desenvolvimento de software. Tanto o GitHub quanto o GitLab oferecem ferramentas integradas para facilitar a revisão de código. Os desenvolvedores podem comentar em linhas específicas, sugerir alterações e aprovar ou rejeitar solicitações de mesclagem, tudo dentro da interface da plataforma.

Integração contínua (CI) e implantação contínua (CD): ambas as plataformas oferecem integração contínua e implantação contínua integradas. Isso significa que, sempre que há uma alteração no repositório, a CI/CD automatiza os processos de compilação, teste e implantação, garantindo uma entrega mais rápida e consistente do software.

GitHub e GitLab

Visibilidade e transparência: as plataformas fornecem uma interface amigável para visualizar o histórico do repositório, entender quem fez quais alterações e quando. Isso oferece uma visão clara do progresso do projeto, contribuições individuais e histórico de problemas.

Segurança e controle de acesso: ambas as plataformas oferecem recursos avançados de segurança e controle de acesso. Os administradores podem gerenciar permissões, configurar políticas de segurança e garantir que apenas as pessoas autorizadas tenham acesso ao código.

Vamos
fazer uma
atividade

Estudo de caso: João enfrenta problemas em seus softwares

Contexto: João, um engenheiro de software experiente e proprietário de uma empresa especializada no desenvolvimento de aplicativos, enfrenta desafios significativos. Seus programadores trabalham simultaneamente no mesmo aplicativo, acelerando o processo da empresa. No entanto, essa abordagem resulta frequentemente em reuniões prolongadas, nas quais os programadores buscam integrar os códigos desenvolvidos em uma única solução coesa.

Convicções pessoais: João tem fortes convicções de que os programadores estão utilizando os métodos de controle de versão manual de forma errada.

Relato fictício elaborado especialmente para o curso.

Vamos
fazer uma
atividade

Estudo de caso: João enfrenta problemas em seus softwares

Problema: você é um consultor e foi contratado pela empresa de João. Agora, justifique de forma coesa e objetiva quais melhorias sugere para a empresa. Convença João de que implementar o controle de versão digital com GitLab ou GitHub é o melhor caminho.

Atividade: escreva a resolução do problema, de forma reduzida e objetiva, baseando-se no que foi visto em aula. Envie suas respostas pelo AVA.

Relato fictício elaborado especialmente para o curso.



© Getty Images

O que nós
**aprendemos
hoje?**

Hoje desenvolvemos:

- 1** O entendimento sobre o Git, seu funcionamento e sua origem.
- 2** A comparação entre diferentes tipos de aplicações que utilizam o Git.



Saiba mais

Nestes cursos, você aprenderá habilidades fundamentais de Git e Github, importantes para desenvolvedores de software. Serão abordados o uso de Git e Github como sistemas de controle de versão, incluindo gestão de *merges*, conflitos, *branches* e técnicas como Git Flow.

Além disso, você explorará recursos avançados, como *cherry-pick*, *Bisect*, Git Desktop, além de entender *hooks*, eventos e *pull requests*, compreendendo as diferenças entre Git e Github no controle de versões de projetos.

ERAHARDT, Fernanda. Plano de estudo. Git e Github para sobrevivência. **Alura**, 2022. Disponível em:

<https://cursos.alura.com.br/git-e-github-para-sobrevivencia-fernanda-erahardt-1666909437927-p471328>. Acesso em: 16 jan. 2024.

LIMA, Guilherme. Plano de estudo. Git e GitHub. **Alura**, 2023. Disponível em: <https://cursos.alura.com.br/git-e-github-guilhermelima-1681997832879-p579648>. Acesso em: 16 jan. 2024.

Referências da aula

AQUILOS, A.; FERREIRA, R. **Controlando versões com Git e GitHub**. São Paulo: Casa do Código, 2014.

CHACON, S.; STRAUB, B. **Pro Git**. [S.l.]: Apress, 2014. *E-book*. Disponível em: <https://git-scm.com/book/pt-br/v2>. Acesso em: 16 jan. 2024.

Identidade visual: imagens © Getty Images

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**